BlackJack 実装仕様書

0. 文書情報

項目	内容
文書名	BlackJack 実装仕様書（最終完成版・ライブラリ委譲拡大版）
対象	認証済みユーザー向け **1 対 1 ブラックジャック**
最大参加人数	**利用者 1 名／卓**（※コンピューターディーラーは人数に含めない）
対戦形式	**利用者（プレイヤー） vs コンピューターディーラー**
ディーラー	**コンピューター（ヒューリスティック AI 基本戦略）**
通信	HTTP + WebSocket
正本	PostgreSQL
補助	Redis
アーキテクチャ	Clean Architecture
ブラックジャックのルール計算	外部ライブラリ（Adapter 経由）。手評価に加え、**1 ラウンド内の進行アルゴリズム・勝敗判定の算出を委譲**
ゲーム進行（システム）	**永続化・通知・再接続・冪等・整合性制御・利用者ターン締切は自作**。ルールエンジンはライブラリ
初版スコープ	Hit / Stand / ルーム / 再戦 / 切断復帰 / CI/CD
非対象	Split / Double / Insurance / Bet / チップ管理 / 観戦モード

1. 目的

本システムは、認証済みユーザーが同一バックエンドに接続し、**1 卓あたり利用者 1 名**で、**コンピューターディーラー**を相手にリアルタイムでブラックジャックをプレイできる Web アプリケーションを提供する。

本システムの設計原則は以下とする。

- ブラックジャックの**進行・勝敗算出を含むルール計算**は外部ライブラリに委譲する
- **ディーラー AI とライブラリ呼び出し順のオーケストレーションは UseCase に集約**する

- ルーム管理、再接続復元、状態遷移、同期配信、永続化、冪等制御は自作する
- ゲーム状態の真実値は常に PostgreSQL とする
- WebSocket は通知・追従同期のために用い、正本にしない
- 更新系処理は必ず 冪等性、状態チェック、競合制御 を通す
- CI/CD パイプライン上で 品質ゲートを通過した成果物のみ をデプロイする

---

2. 技術スタック

2.1 アプリケーション

区分	採用
フロントエンド	React + TypeScript + Vite
バックエンド	Go 1.24
HTTP	Echo v4
WebSocket	gorilla/websocket
ORM	GORM
DB	PostgreSQL 15+
Redis	go-redis v9
セッション	Cookie Session + PostgreSQL
CSRF	Double Submit Cookie
FEテスト	Vitest / Testing Library
BEテスト	Go testing / testify

2.2 ブラックジャックライブラリ

依存固定:

- github.com/ethanefung/blackjack を go.mod で最新以外のコミットハッシュまたはタグに pin する
- 間接依存 github.com/ethanefung/cards も合わせて pin する

用途	採用
ルールエンジン（手評価・**進行・勝敗算出**）	`github.com/ethanefung/blackjack`
間接依存	`github.com/ethanefung/cards`
Adapter 役割	`HandEvaluator` + **`RoundEngine`**（名称は実装に合わせてよい）

2.3 CI/CD / 運用

区分	採用
CI/CD	GitHub Actions
デプロイ先	**Vercel（GitHub Actions から直接デプロイ）**
コンテナ	docker
レジストリ	GHCR
シークレット管理	**GitHub Secrets + Vercel Environment Variables**
監視	**Vercel Observability + アプリケーションログ**
負荷試験	k6

3. システム概要

3.1 プレイモデル

- 利用者はコンピューターディーラーと対戦する
- **1 ルーム（卓）あたり人間プレイヤー最大 1 名**
- **プレイヤー同士の相互対戦は存在しない**
- 1 ラウンドは配札開始から結果確定まで
- ラウンド終了後は再戦投票フェーズへ遷移する

3.2 状態正本

- 真実値は PostgreSQL
- Redis はレート制限、Pub/Sub、接続補助のみ

- リアルタイム通信は Go バックエンドの WebSocket サーバーにより処理される。
- Redis は Pub/Sub を補助的に利用する場合があるが、リアルタイム通信の主体は WebSocket とする。
- Vercel Functionsはステートレスであり、長時間接続は持たない
- 再接続時は DB 正本から状態を再構築して送信する

---

4. 設計原則

4.1 Clean Architecture

層	責務
Controller	HTTP/WS 入出力、認証、DTO 変換、レスポンス生成
UseCase	トランザクション、ルーム・セッション状態遷移、整合性保証、**ディーラー AI の適用**、**ライブラリ呼び出し順の統制（唯一のオーケストレータ）**
Domain	副作用なしの整合性・遷移可否。**数値計算は直接ライブラリを呼ばずポートに委譲**し、ルールの正・不変条件を担う
Infrastructure	PostgreSQL、Redis、Session、WS 接続管理、**Blackjack Adapter（HandEvaluator + RoundEngine）**

4.2 原則

- Domain は副作用を持たない
- DB更新は UseCase のみ
- Controller は業務ロジックを持たない
- WS受信から直接DBを更新しない
- WS通知はコミット後のみ
- クライアントは常にサーバー正本を正とする

4.3 ディーラー AI

目的:

コンピューターディーラーの Hit / Stand を、ヒューリスティック基本戦略に基づいて決定する。

入力例:

- ディーラーの現在手札
- 公開済みカード情報
- 現在の session 状態
- 仕様で固定されたルール（6.1）

出力例:

- `HIT`
- `STAND`

ルール:

- ディーラー AI は **6.1 の必須ルールを破ってはならない**
- ディーラー AI の判断結果は **UseCase が最終ガード**する
- **適用タイミング・Tx 境界は UseCase が管理**する
- AI はあくまで行動選択を補助し、DB 更新・version 更新・WS 通知は直接行わない

4.3.1 ヒューリスティック AI の定義

本仕様における **ヒューリスティック AI** とは、機械学習モデルや外部推論 API に依存せず、**手札の評価結果（スコア・ソフト可否・バスト可否）に対する決定規則の優先順位付き集合**により、コンピューターディーラーの **Hit / Stand** を決定する方式をいう。

- 同一の内部状態に対して **決定は一意に定まる**（再現性・テスト容易性のため）。
- **6.1** の必須ルールを満たさない行動を返してはならない。
- ディーラー標準ルールに従い、**利用者の手札情報を行動決定の入力に用いない**（将来ルールを変える場合は別途仕様化する）。

4.3.2 レイヤ別責務（Clean Architecture）

層	責務
UseCase	ディーラーターンにおける **いつヒューリスティックを呼ぶか**、**ライブラリ呼び出し順**、**トランザクション境界**、**永続化・version・通知**のオーケストレーション。**ヒューリスティックの結果に対する最終ガード**（6.1 適合確認）を必ず行う。
Domain	行動の列挙（例: Hit / Stand）や **6.1 に基づく不変条件**の宣言に留める。外部ライブラリ具象に依存しない。
Infrastructure	`HandEvaluator` および `RoundEngine` 等の **Adapter 実装**。副作用のある DB・WS は本層または UseCase 経由で行う。

ヒューリスティック本体は副作用を持たない **純粋関数**または `DealerPolicy` **（または同等名称）インタフェースの実装**とし、**UseCase から依存注入**する。I/O を伴わないため、パッケージ配置は UseCase 配下のサブパッケージでも、Domain が定義するポートの実装を Infrastructure に置く形でもよい。

4.3.3 基本戦略表（ディーラー・決定版）

前提: 6.1「17 未満 Hit、17 以上 Stand」「Soft17 は Stand」。

条件（評価後）	行動	備考
`isBust == true`	（ドロー不要）	ラウンドは結果フェーズへ。ここでは「次のディーラー行動」自体が存在しない
`handValue >= 21`（非 Bust なら 21 のみ）	`STAND`	実質 21 で確定
`handValue >= 17`	`STAND`	Hard 17 / Soft 17 いずれも **Stand**
`handValue == 17 && isSoft`	`STAND`	**Soft 17 = Stand**（6.1 明示。上行と重複だが監査・可読性のため残す）
`handValue < 17`	`HIT`	16 以下（hard / soft 問わず）

論理の簡約形（実装ヒューリスティック）

1. `IsBust(hand)` → 以降の Hit/Stand 判断を行わない（結果処理へ）。
2. `Value(hand) > 21` は Bust と同義。
3. `Value(hand) >= 17` → `STAND`。
4. それ以外 → `HIT`。

4.3.4 決定アルゴリズム（擬似コード）

UseCase 内の **最終ガード**として使う想定。ライブラリ呼び出し前後どちらでも使用可だが、**永続化前に必ず一度通す**。

```
// NextDealerAction: 6.1 準拠の次アクション
func NextDealerAction(ev HandEvaluator, dealer []StoredCard)
(action HitOrStand, terminal bool) {
    if ev.IsBust(dealer) {
        return Stand, true // 以降ディーラーはカードを引かない（結
果処理）
    }
    v := ev.Value(dealer)
    if v == 17 && ev.IsSoft(dealer) {
        return Stand, false // Soft 17 明示ガード
    }
    if v >= 17 {
        return Stand, false
    }
    return Hit, false
}
```

- `terminal == true` のときは「ディーラーターン内でこれ以上ドローしない」を意味するが、状態機械の RESULT への遷移は別 Tx（10）。
- `IsSoft` の明示分岐は `v >= 17` で既に Stand になる場合と重複するが、**監査・可読性**のため残してよい。

4.3.5 導入手順（実装の正規手順）

1. **手札評価ポートの確定** `HandEvaluator`（`Value` / `IsSoft` / `IsBust` 等）を Adapter 経由で利用可能にする（5.3）。
2. **決定関数の実装**
   入力: ディーラー現在手（正本から復元した表現）。出力: `HIT` または `STAND`（実装で表記を固定する）。論理は 4.3.3〜4.3.4 に従い、**6.1 と一致**させる。

3. **UseCase への組み込み**
   ディーラーターンにおいて **各ドロー前**（またはライブラリのディーラー行動決定 API 呼び出し**直後**）に上記関数を実行し、**永続化前に最終確認**する。
4. **ライブラリとの整合**
   ライブラリが `DecideDealerAction` 等を返す場合、返却値をヒューリスティックまたは 6.1 に基づき正規化し、矛盾時は **本仕様（6.1・5.5）を優先**する。
5. **テスト**
   代表状態（Hard 16、Hard 17、Soft 17、Bust 等）に対する **表駆動テスト**を行い、決定性を検証する（4.3.6・21）。

4.3.6 ライブラリ連携パターン（ディーラー Hit/Stand）

ディーラーの **Hit / Stand の最終可否**は、以下方式で決定する。

ただし、**正本は常に 6.1 および 4.3 系ルールとする**。

ライブラリの返却値は**そのまま正本として扱わない**

- 次アクションは 4.3 の決定ロジック（または同等）のみで決定する
- ライブラリは以下の用途に限定する
  - 手札評価
  - Hit後の手札更新
  - スコア計算
- `RoundEngine.DecideDealerAction` は **使用しない**

共通ルール

- DB 正本の行動履歴・outcome は **6.2 の列挙に従う**
- Controller / WS ハンドラでルール判定を行わない
- **UseCase が唯一のオーケストレーター**

4.3.7 禁止事項

- **Controller** または **WebSocket 受信ハンドラ**内で、ディーラーの Hit/Stand を決定しない。

- ヒューリスティックの結果として **ランダムに Hit/Stand を選択しない**（基本戦略の再現性と監査に反する）。
- ヒューリスティック関数内で **DB・Redis・WebSocket を直接呼び出さない**（純粋性を維持する）。

5. 外部ライブラリ利用方針

注（厳密整合）：

「Dealer の行動決定」は、ライブラリが次の行動を返却する機能を指す。

ただしゲーム上の正しい行動は**6.1・4.3**に従う。

5.1 利用範囲

外部ライブラリは以下に限定する。

項目	内容
手札スコア計算	ライブラリ委譲
Ace解釈（1/11）	ライブラリ委譲
Blackjack判定	ライブラリ委譲
Bust判定	ライブラリ委譲
Soft判定	ライブラリ委譲
プレイヤー手更新（Hit/Stand）	ライブラリ委譲
ディーラー進行	ライブラリ委譲（手札更新・進行ステップのルール計算）。ただし **Hit/Stand の最終判断は 4.3.6 に従い、6.1・4.3 を正とする**
勝敗判定	ライブラリ委譲（Adapterで正規化）

5.2 利用禁止

以下は**ブラックジャックのゲームルールそのものとしては扱わず**、システム側で責務を持つ。

- ルーム管理
- ラウンド開始トリガ
- セッション作成・終了

- 山札の永続化
- 配札結果・進行結果の永続化
- WebSocket 同期
- 再接続復元
- 冪等制御
- version / 排他制御
- タイムアウト監視
- 認証・認可
- 再戦受付・再戦確定
- 監査ログ保存

補足

- **ブラックジャックのゲーム進行そのものはライブラリに委譲する**
- これには少なくとも以下を含む
  - 手札評価
  - Hit 後の手札更新
  - Stand 後の進行判定
  - Dealer の行動決定
  - Dealer の進行
  - Bust / Blackjack 判定
  - 勝敗算出
- システム側は、ライブラリが返した結果を **DB 正本へ保存し、必要に応じて 6.2 の outcome に正規化**して扱う
- **UseCase はルール計算を再実装しない**
  役割は、ライブラリ呼び出し、Tx 制御、整合性保証、永続化、通知のオーケストレーションに限定する

5.2.1

- ディーラー Hit / Stand の正は **6.1 および 4.3**
- `DecideDealerAction` は必須ではない
- 使用する場合は **パターン B** を適用し、必ず正規化する
- UseCase は**完全なルールエンジンを再実装しない**
- 4.3 は「ルール」ではなく **ポリシー／ガード層**

※ ディーラーの Hit / Stand の最終可否については、本補足の列挙（「Dealer の行動決定」等）より **§4.3.6 および §5.2.1** を優先して解釈すること。ライブラリは進行・手札更新等の計算に用いるが、**正本の行動決定は §4.3 の決定ロジック（または同等）および §6.1** とする。

5.3 Adapter 契約

```
type HandEvaluator interface {
    Value(hand []StoredCard) int
    IsBlackjack(hand []StoredCard) bool
    IsBust(hand []StoredCard) bool
    IsSoft(hand []StoredCard) bool
}

type RoundEngine interface {
    ApplyPlayerHit(hand []StoredCard, draw StoredCard) (nextH
and []StoredCard, err error)
    DecideDealerAction(dealerHand []StoredCard) (action strin
g, err error)
    ResolveOutcome(playerHand []StoredCard, dealerHand []Stor
edCard) (outcome string, err error)
}
```

実装メモ（Adapter）：

- `DecideDealerAction` は **インタフェース上は存在し得るが、ディーラーの次手の最終決定には用いない**（§4.3.6）。本仕様では **§4.3 の決定ロジック（または同等）のみ**

で次手を確定する。

- ライブラリ実装が当該メソッドを提供する場合でも、UseCase 側では **返却値を正本として保存・通知に用いず**、必要なら §5.5 に従い **§6.1・§4.3 に正規化した結果のみ**を永続化・配信する。
- 新規 Adapter 実装時は、`DecideDealerAction` を呼ばない、または呼んでも結果を捨てて §4.3 で再決定する、のいずれかとし、**「仕様に載っているから必ず呼ぶ」誤読を防ぐ**こと。

5.4 判定ルール

- IsBlackjack : len(hand) == 2 && Value(hand) == 21
- IsBust : Value(hand) > 21
- IsSoft : Ace を11として評価している状態

5.5 ライブラリ挙動差分の優先順位

外部ライブラリの挙動が本仕様と矛盾する場合、本仕様を優先する。

特にディーラー進行は本仕様の `Soft17 = Stand` を優先し、必要なら Adapter 内で補正する。

outcome は常に 6.2 の列挙へ正規化し、ライブラリの勝敗型をそのまま正本にしない。

最終的な保存値・通知値・監査値は、本仕様に従って UseCase が決定する。

6.1 基本ルール

- 初期配布はプレイヤー 2 枚、ディーラー 2 枚
- 21 超えで Bust
- 初期 2 枚で 21 のみ Blackjack
- 3 枚以上で 21 は通常の 21
- 数札は額面、J/Q/K は 10、A は 1 または 11
- ディーラーは 17 未満で Hit、17 以上で Stand
- Soft17 は Stand

- **コンピューターディーラーは 4.3 のヒューリスティック戦略を適用しつつ、本条の必須ルールを満たす**

6.2 勝敗優先順位

1. プレイヤー Bust → LOSE
2. ディーラー Bust かつプレイヤー非 Bust → WIN
3. プレイヤー BJ かつディーラー非 BJ → WIN
4. ディーラー BJ かつプレイヤー非 BJ → LOSE
5. 両者 BJ または同点 → PUSH
6. その他は点数比較

---

7. 永続化モデル

7.1 テーブル一覧

- rooms
- room_players
- game_sessions
- player_states
- dealer_states
- action_logs
- round_logs
- rematch_votes （本完成版で追加確定）

7.2 rooms

役割:

- ルーム永続単位
- 粗粒度状態（ロビー表示・参加可否・開始可否）

- 現在の進行セッション参照

主カラム:

- id
- host_user_id
- status
- current_session_id
- created_at
- updated_at

補足:

rooms は楽観ロック用の version 列を持たない。

ルーム作成・参加・退出など「セッション未作成」の更新では、競合制御用の version は存在しない。

ルームに関する競合制御・更新整合性は game_sessions.version を正とする（セッションが存在する操作のみ）。

rooms.status はロビー表示・参加可否・開始可否判断用の粗粒度状態とする。

ラウンド内の詳細進行は game_sessions.status を正とする。

7.3 room_players

主カラム:

- `room_id`
- `user_id`
- `seat_no`
- `status`
- `joined_at`
- `left_at`

ルール:

- **同一 Room 内の人間プレイヤーは最大 1 名**

- 2 人目の `join` は `room_full`
- `seat_no` は **1 固定**でよい
- 同一 Room 内で `ACTIVE / DISCONNECTED` の人間プレイヤー重複は禁止
- `LEFT` は進行対象外
- 再 join は新規参加扱いとしてよいが、本版では同時複数 seat は持たない
- 複数 seat 前提のルールは **本版では未使用**

7.4 game_sessions

主カラム:

- id
- room_id
- round_no
- status
- version（bigint、楽観ロック。新規セッションは 1 から開始）
- deck
- draw_index
- turn_seat
- turn_deadline_at
- result_snapshot
- rematch_deadline_at
- created_at
- updated_at

補足:

本システムの楽観ロックは game_sessions.version のみ（8.1 参照）。

7.5 player_states

役割:

参加者ごとの手札と進行状態（単一 status）

主カラム:

- session_id
- user_id
- seat_no
- hand
- status （ACTIVE | STAND | BUST | BLACKJACK | DISCONNECTED | LEFT）
- outcome （RESULT 確定後のみ）
- final_score （RESULT 確定後のみ。任意）

制約:

- UNIQUE(session_id, user_id)
- UNIQUE(session_id, seat_no)

禁止:

- is_stand / is_bust のような重複表現（status と二重管理しない）

7.6 dealer_states

主カラム:

- session_id
- hand
- hole_hidden
- final_score

7.7 action_logs

役割:

- 冪等性

- 監査
- 再送応答保存

主カラム:

- session_id
- actor_type
- actor_user_id
- target_user_id
- action_id
- request_type
- request_payload_hash
- response_snapshot

一意制約:

- UNIQUE(session_id, actor_user_id, action_id)

成功応答のみ response_snapshot に保存する。

7.8 rematch_votes

主カラム:

- id
- session_id
- user_id
- agree
- created_at
- updated_at

制約:

- UNIQUE(session_id, user_id)

意味:

- 同一ユーザーの再投票は upsert で上書き
- 最終票のみ有効

7.9 `round_logs`

- 役割: ラウンド単位の**監査用・不変ログ**（正本は `game_sessions` 等）。
- 列例: `id`, `session_id`, `round_no`, `result_payload`（JSONB）, `created_at`。
- `UNIQUE(session_id, round_no)`。9.3.8 の **Tx コミット成功後**に append のみ。

8. version / action_id / request_id

8.1 version

本システムの楽観ロック用バージョンは game_sessions.version のみとする。

rooms.version は保持しない（または保持してもAPI/WSの競合制御には使用しない）。

更新系処理（HIT / STAND / 自動STAND / Dealer Draw / Dealer終了 / REMATCH確定）成功時にのみ

game_sessions.version = version + 1 。

read 操作、再接続、同一 action_id の冪等再送では増加させない。

8.2 action_id（業務冪等キー）

更新系イベントは payload.action_id 必須。

同一 (session_id, actor_user_id, action_id) が既に成功している場合:

- request_payload_hash が同一なら前回成功レスポンスを返す（冪等成功）
- 異なるなら duplicate_action を返す

8.3 request_id（通信相関キー）

request_id は通信上の対応付けキーであり、冪等性判定には使わない。

再送時に新しい request_id を用いてよい。

---

9. 状態遷移仕様

9.1 Room 状態

- `WAITING`
- `READY`
- `PLAYING`

判定ルール:

- **人間プレイヤー 0 名** → `WAITING`
- **人間プレイヤー 1 名 かつ 進行中 session なし** → `READY`
- `current_session_id` が存在し、`session.status` が `DEALING / PLAYER_TURN / DEALER_TURN / RESULT / RESETTING` のいずれか
  → `PLAYING`

最優先:

- 進行中 session あり → `PLAYING`

次:

- 1 名・session なし → `READY`
- 0 名 → `WAITING`

補足:

- 本版では **多人数 READY 判定や allow_single_player_start の分岐は使用しない**

9.2 Session 状態

- DEALING
- PLAYER_TURN
- DEALER_TURN
- RESULT
- RESETTING

9.3 状態遷移表

9.3.1 ルーム作成

項目	内容
前提	認証済み
実行主体	USER
更新	rooms 生成、room_players に host 参加
結果状態	WAITING
version	rooms に version 列は持たない（7.2 参照）
失敗	unauthorized / internal_error

9.3.2 ルーム参加

項目	内容
前提	`room.status = WAITING or READY`
実行主体	USER
更新	`room_players` 追加
結果状態	人数で `WAITING/READY` 再計算
version	ルーム一覧キャッシュ無効化などが必要なら `rooms.updated_at` のみ更新してよい
競合制御	version の増分は行わない（ゲーム未開始のため session が無い）
失敗	`room_not_found / room_full / invalid_game_state`

追記:

- **既に人間プレイヤーが存在する場合は `room_full`**

9.3.3 開始

項目:

前提:

- host
- room.status = READY
- 進行中 session なし（current_session_id が null）

- クライアントは version を送らない（セッション未作成のため競合制御対象が存在しない）

実行主体: USER

更新:

- game_sessions を新規作成（version = 1）
- deck 保存、配札、dealer_states 作成、player_states 作成
- rooms.current_session_id を新 session id に更新

結果状態:

- session.status = DEALING →（配札完了後）PLAYER_TURN
- room.status = PLAYING

失敗:

- forbidden / invalid_game_state

9.3.4 HIT

項目	内容
前提	`session.status = PLAYER_TURN`、自ターン、未Stand、未Bust
実行主体	USER
更新	1 枚 draw、hand 更新、Bust 判定
結果状態	`PLAYER_TURN` または `DEALER_TURN`
version	+1
失敗	`not_your_turn / invalid_game_state / version_conflict / duplicate_action`

注記:

- **人間プレイヤーは 1 名のみ**であるため、seat 巡回は行わない
- 手の更新・Bust 等の**ルール適用はライブラリ可**

9.3.5 STAND

項目	内容
前提	`session.status = PLAYER_TURN`、自ターン、未Stand、未Bust
実行主体	USER
更新	`player_states.status = STAND`
結果状態	**`DEALER_TURN`**
version	+1
失敗	`not_your_turn / invalid_game_state / version_conflict / duplicate_action`

注記:

- **利用者のみがプレイヤーターンを持つ**
- `STAND` 後は **人間間の turn 移動はなく、そのまま** `DEALER_TURN` に入る
- 手の更新・状態判定などの**ルール適用はライブラリ可**

9.3.6 自動STAND

項目	内容
前提	turn_deadline_at 到達、未操作
実行主体	SYSTEM
更新	player_states.status = STAND、action_logs 記録（actor_type=SYSTEM）
結果状態	**DEALER_TURN（本版は 1 対 1 のため PLAYER_TURN は存在しない）**
version	+1
失敗	internal_error

9.3.7 Dealer Draw

項目	内容
前提	session.status = DEALER_TURN、dealer score < 17
実行主体	SYSTEM
更新	dealer hand に1枚追加、draw_index 更新
結果状態	DEALER_TURN 継続

項目	内容
version	+1
失敗	internal_error

9.3.8 Dealer Turn 終了

項目	内容
前提	dealer score >= 17
実行主体	SYSTEM
更新	hole_hidden = false、勝敗確定、result_snapshot 保存、round_logs 保存
結果状態	RESULT
version	+1
失敗	internal_error

9.3.9 RESULT → RESETTING

項目	内容
前提	勝敗確定直後
実行主体	SYSTEM
更新	rematch_deadline_at = now + 30s
結果状態	RESETTING
version	+1
失敗	internal_error

9.3.10 REMATCH 成立

項目	内容
前提	session.status = RESETTING 、対象者全員 agree = true
実行主体	SYSTEMのみ（投票受信処理または締切ジョブ内で最終確定）
更新	新しい game_sessions を作成（次ラウンド）、 rooms.current_session_id を新セッションへ更新、配札準備

項目	内容
結果状態	rooms.status = PLAYING 、新 session.status = DEALING （直後に PLAYER_TURN ）
version	新セッションは version = 1 で開始（旧セッションは完了済み）
失敗	invalid_game_state / internal_error

9.3.11 REMATCH 不成立

項目	内容
前提	session.status = RESETTING 、締切到達または否認票あり
実行主体	SYSTEM
更新	rooms.current_session_id = null 、 rooms.status を有効参加者数で再計算
結果状態	WAITING または READY
version	ルーム再計算のみ（セッションversionは増やさない）
失敗	internal_error

10. Dealer進行のトランザクション境界

10.1 方針

Dealer の1ドローを1トランザクションとする。

理由:

- 1ドロー1Tx にすることで、観測可能状態と version 増分が一致する
- 障害時の中途状態が明確になる
- 再送・再開がしやすい

10.2 Dealer進行アルゴリズム

1. `session.status = DEALER_TURN` を確認
2. **UseCase がディーラー AI（4.3）とライブラリを用いて次手を決定**する
3. 次手が `HIT` の場合は 1 枚 draw

4. その 1 ドロー分だけ Tx で保存
5. `game_sessions.version` を +1（Dealer の 1 ドロー = 1 回の観測可能更新）
6. commit 後に `ROOM_STATE_SYNC` 配信
7. 次ドロー要否を再評価
8. 次手が `STAND`、または `dealer score >= 17` なら終了確定 Tx へ進む

10.3 Dealer終了トランザクション

終了時は別Txで以下を行う。

- hole_hidden = false
- 各プレイヤー outcome 計算
- final_score 確定
- result_snapshot 保存
- round_logs 保存
- session.status = RESULT
- game_sessions.version = game_sessions.version + 1

10.4 禁止事項

- Dealer ターン全体を 1つの長大トランザクションにしない
- goroutine 内メモリ状態だけで Dealer を進めない
- commit 前に WS を送らない

---

11. ゲーム進行仕様

11.1 プレイヤーターン

- **プレイヤーターンを持つのは利用者のみ**
- 自ターン制限は 15 秒
- 締切までに未操作なら自動 STAND

- `status` が `STAND / BUST / BLACKJACK` ならプレイヤーターン終了
- seat 昇順や複数人スキップの概念は **本版では適用しない**

11.2 山札

- 1ラウンドごとに 52 枚 1 デッキ
- ラウンド開始時にシャッフル
- シャッフル済み順序を game_sessions.deck に保存
- 消費位置は draw_index で管理
- 再接続時は deck + draw_index で復元可能でなければならない
- ルール適用はライブラリ可。ただし、山札の保存・消費位置管理・再接続復元・Tx 境界はシステム側で管理する

---

12. 再戦仕様

12.1 対象者

RESETTING開始時点でACTIVEまたはDISCONNECTEDの**人間プレイヤー**

ディーラーは対象外。

12.2 投票方式

- 更新系 WS イベント REMATCH_VOTE
- HTTP fallback は持たない

12.3 投票内容

- agree = true | false
- 1人1票
- 再投票は上書き

12.4 締切

- RESULT → RESETTING 遷移時点から 30秒
- rematch_deadline_at に保存

12.5 成立条件

- **対象者（人間プレイヤー）の全員が `agree = true` で即成立**
- **1卓1人のため、その1名の `agree = true` で成立**
- 締切時に賛成でなければ不成立
- 未投票は `false` とみなす

12.6 再戦確定処理

再戦確定は SYSTEM が一度だけ実行できる。

排他は session_id 単位で SELECT ... FOR UPDATE （または advisory lock）。

確定アルゴリズム:

1. session.status == RESETTING を確認
2. 対象者一覧確定
3. 票集計（未投票は false）
4. 全員 true なら「成立」、それ以外「不成立」
5. 成立/不成立を単一TXで確定

一度確定後の REMATCH_VOTE は invalid_game_state 。

12.7 競合ケースの扱い

ケース1: 締切直前投票と締切ジョブ競合

- lock を取った側のみ確定可能
- 後続処理は session.status 確認で no-op

ケース2: 再戦成立直後に leave

- 再戦成立後、次ラウンド開始前なら leave 可

- leave により有効参加者が1名以下になった場合、room.status を WAITING に再計算

ケース3: host が DISCONNECTED

- host 権限は保持
- ただし再戦後の開始操作は host 接続有無とは別に、認証済み host の操作でのみ許可
- host が LEFT なら最小 seat の有効参加者へ委譲

ケース4: RESETTING 中の join

- 禁止
- invalid_game_state

---

13. 切断・再接続仕様

13.1 切断時

- WS切断検知で room_players.status = DISCONNECTED
- 席は維持
- 自ターン中は締切まで待つ
- 締切まで未操作なら SYSTEM による自動 STAND
- action_logs に actor_type = SYSTEM 、 target_user_id = 対象ユーザー を保存

13.2 再接続時

- 参加者であれば接続可
- 新接続確立時、旧接続が存在すれば旧接続を close
- 新接続直後に ROOM_STATE_SYNC を1回送信
- 内容は DB 正本から再構築した state
- 再接続では version を増やさない

13.3 多重接続の扱い

- 同一 room_id + user_id に対して最新接続のみ有効
- 接続管理レジストリは各インスタンスのメモリに持つ
- 複数インスタンス環境では
  Redis に room:{roomId}:user:{userId}:connection_epoch を短TTLで保持し、最新 epoch のみ受理する
- 古い接続から届いた更新系イベントは forbidden または close
- `connection_epoch` は補助。最終判断は **DB + version + action_id**。

13.4 Rolling Update 中の扱い

WebSocket サーバーの rolling update 中、既存接続の切断は許容する。

クライアントは接続 close を検知した場合、自動的に再接続を行う。

再接続後は以下のいずれかにより状態を復元する：

- GET /api/rooms/{id}
- WebSocket 初回接続時の ROOM_STATE_SYNC

フロントエンドの Vercel デプロイは WebSocket 接続維持責務を持たない。

接続断は異常ではなく、DB 正本から復元可能であることを前提とする。

14. WebSocket 仕様

14.1 接続

GET /ws/rooms/{roomId}

条件:

- Cookie Session 認証
- 参加者のみ接続可能
- 多重接続時は最新のみ有効

本システムの WebSocket は Go バックエンドにより提供される常駐サーバーで処理される。

Vercel は WebSocket 接続を終端しない。

本番構成の正本:

- **フロントエンド**（React 等）は Vercel にデプロイする想定とする。
- **HTTP API および WebSocket（長時間接続）を処理する Go バックエンド**は、Vercel Functions 上では常駐できないため、**別の常駐実行環境**（例: コンテナ常駐ホスト、対応する PaaS、自前 VM 等）にデプロイする。具体的なホスト名・スケール方針・環境変数の一覧は **リポジトリ内の IaC（Terraform 等）およびデプロイ手順書を正本**とし、本 PDF／Notion は要約とする。
- クライアントの WebSocket 接続先 URL は、**上記 Go バックエンドのオリジン**と一致させる（Vercel のオリジンだけでは WS を完結させない）。

運用上の確認事項:

- ローリングアップデート時の切断・再接続は §13.4 のとおり許容し、状態の最終判断は DB（game_sessions.version 等）とする。

14.2 Client -> Server

- HIT
- STAND
- ROOM_SYNC_REQUEST
- PING
- REMATCH_VOTE

14.3 Server -> Client

- ROOM_STATE_SYNC
- ERROR
- PONG

14.4 共通ルール

原則: game_sessions.version の扱いは

読み取り系（同期） と 更新系 で返却形が異なる。

14.4.1 読み取り系・同期系

トリガ	典型	クライアントが送る `version`	`version` がサーバー正本と不一致のとき
WS	`ROOM_SYNC_REQUEST`、再接続直後の初回同期	任意（不明でも可）	**`ERROR` にしない。`ROOM_STATE_SYNC` のみ**で最新 `state.session.version` を返す
HTTP	`GET /api/rooms/{id}`	クエリで送らない（本文どおり）	レスポンスに **現在の** `session.version` を含める

14.4.2 更新系

- HIT / STAND / REMATCH_VOTE
- 既存 session に対する更新

→ `session.version 必須`

→ 不一致は `version_conflict`

クライアント挙動

1. version 破棄
2. 正本取得
3. 再送

14.5 state schema ルール

- players は全員含む
- 自分の hand のみ返す
- 他人は card_count
- dealer は hidden 管理
- 行動可否は my_actions
- result は確定後のみ

14.5.1 ROOM_STATE_SYNC.state

```
{
  "room": {
    "id": "uuid",
    "status": "WAITING|READY|PLAYING"
  },
  "session": {
    "id": "uuid|null",
    "status": "DEALING|PLAYER_TURN|DEALER_TURN|RESULT|RESETTI
NG|null",
    "version": 12,
    "round_no": 3,
    "turn_seat": 1,
    "turn_deadline_at": "2026-01-01T00:00:00Z|null",
    "rematch_deadline_at": "2026-01-01T00:00:30Z|null"
  },
  "dealer": {
    "visible_cards": ["..."],
    "hidden": true,
    "card_count": 2
  },
  "players": [
    {
      "user_id": "uuid",
      "seat_no": 1,
      "status": "ACTIVE|STAND|BUST|BLACKJACK|DISCONNECTED|LEF
T",
      "is_me": true,
      "hand": ["..."],
      "card_count": 2,
      "outcome": "WIN|LOSE|PUSH|BUST|null",
      "final_score": 20
    }
  ],
  "my_actions": {
```

```
    "can_hit": true,
    "can_stand": true,
    "can_rematch_vote": false
  }
}
```

注記:

- `players` の**人間エントリは 1 件が標準**
- `turn_seat` は **1 固定**の例でよい

---

15. HTTP API 仕様

`POST /api/rooms/{id}/join` は、**既に人間プレイヤーが存在する場合** `room_full` を返す

15.1 一覧

```
POST /api/auth/login
POST /api/auth/logout
GET  /api/me
GET  /api/rooms
POST /api/rooms
POST /api/rooms/{id}/join
POST /api/rooms/{id}/leave
GET  /api/rooms/{id}
POST /api/rooms/{id}/start
GET  /api/rooms/{id}/history
```

15.2 共通レスポンス

成功:

```
{
  "success":true,
```

```
  "data": {}
}
```

失敗:

```
{
  "success":false,
  "error": {
    "code":"invalid_input",
    "message":"..."
  }
}
```

15.3 Debug API

```
POST /api/rooms/{id}/reset
```

- 開発専用
- 本番では無効

16. 認証・セッション・セキュリティ

16.1 セッション

- 方式: Cookie Session
- 保存先: PostgreSQL
- TTL: 24時間

Cookie 属性:

- HttpOnly
- Secure
- SameSite=Lax
- Path=/

16.2 CSRF

- csrf_token Cookie を発行
- 更新系 HTTP は X-CSRF-Token 必須
- WS は Cookie 認証

16.3 ログイン

- bcrypt でハッシュ
- login 成功時 session rotate
- session fixation 防止

16.4 ログアウト

- server-side invalidation 必須
- Cookie削除のみは禁止

16.5 認可

- room 参加者のみ閲覧可
- start は host のみ
- join は WAITING / READY のみ
- leave は参加者のみ
- history は参加者のみ

16.6 レート制限

- Token Bucket
- capacity = 20
- refill = 5/sec
- Redis 使用

- HTTP / WS 両方適用

---

17. 整合性制御

17.1 三層防衛

1. action_id（冪等）
2. 状態チェック
3. version（競合制御）

17.2 処理順

1. 認証
2. レート制限
3. 読み取り
4. action_id確認
5. 状態チェック
6. 排他取得
7. 更新Tx
8. action_logs保存
9. commit
10. WS通知

17.3 排他ポリシー

- session単位で排他
- SELECT FOR UPDATE 推奨
- advisory lock 可
- version check 必須

17.4 フェーズとの関係

三層防衛（冪等・状態チェック・version）は

game_sessions 存在後の全更新系に適用する

Phase1で以下を保証する：

- HIT / STAND / Dealer / Result
- 冪等
- version
- 排他制御
- 再接続復元

---

18. CI/CD 仕様

18.1 ブランチ戦略

ブランチ	用途
main	本番
develop	開発
feature/	機能
hotfix/	緊急

ルール:

- main直push禁止
- PR必須
- review必須

18.2 環境

環境	用途
local	開発
ci	テスト

環境	用途
staging	検証
production	本番

18.3 CI（PR時）

必須:

1. frontend lint
2. frontend test
3. frontend build
4. backend vet
5. backend test
6. backend build
7. integration test
8. container build
9. security scan

18.4 Integration Test

- PostgreSQL / Redis 起動
- room作成
- join
- start
- hit / stand
- dealer
- result
- rematch
- reconnect

- duplicate_action
- version_conflict

18.5 Security Scan

- gosec
- npm audit
- trivy
- gitleaks

重大NG → deploy禁止

18.6 CD

staging

1. build
2. migration
3. **GitHub Actions から Vercel へ直接 deploy**
4. health check

production

条件:

- staging 成功
- manual approval

18.7 Smoke Test

- /health 200
- /api/me
- room作成
- WS接続
- state一致

18.8 migration

- 自動実行禁止
- pipelineで実行
- backward compatible

18.9 migration失敗

- deploy中断
- 旧バージョン維持

18.10 rollback

- image戻す
- DB rollbackなし

18.11 WS + rolling update

- 切断許容
- 再接続必須
- DBから復元

18.12

integration = 実DB/Redis で API/WS テスト、security = gosec/audit/trivy/gitleaks を失敗連動。`echo` のみは **18.3 未達**

19. GitHub Actions

掲載 YAML はスケルトン。

`integration/security` の `echo` はプレースホルダで Required 不可。`checkout` ・ `services` 等は正本 workflow で補完する。正本は repo の YAML とする。

正本 workflow には Vercel デプロイジョブを含める。

20. 監視・観測性

フロントエンドの監視は **Vercel Observability** を利用する。

WebSocket および API サーバーの監視は、各バックエンド基盤のログおよびメトリクス基盤を利用する。

Fly.io Metrics に依存する記述は本版では使用しない。

20.1 必須ログ項目

- timestamp
- level
- request_id
- action_id
- room_id
- session_id
- user_id
- actor_type
- request_type
- session_version_before
- session_version_after
- latency_ms
- result
- error_code

20.2 メトリクス

- HTTP request count
- HTTP p95 latency
- WS send latency p95
- active ws connections

- reconnect count
- version_conflict count
- duplicate_action count
- auto_stand count
- dealer_draw_count
- room_count
- session_count
- deploy_success_count
- deploy_failure_count

20.3 アラート条件

- 5xx 急増
- WS reconnect 急増
- DB 接続枯渇
- migration failure
- deploy failure
- health check failure
- version_conflict 異常増加

20.4 監査ログ

以下は追跡可能にする。

- start
- hit
- stand
- auto-stand
- rematch vote

- host 移譲
- room leave
- reconnect epoch 更新

---

21. テスト要件

21.1 Domain

- 2枚 A + 10 は Blackjack
- 3枚 7 + 7 + 7 は通常21
- 22以上は Bust
- Dealer A + 6 は Soft17 で Stand
- 勝敗優先順位が仕様どおり
- ライブラリ委譲時は、**Adapter 正規化後の outcome が 6.2 の列挙どおり**であることを検証する
- 表駆動テスト必須
- 決定性保証（同一入力→同一出力）

21.2 UseCase

- HIT/STANDの正常系・異常系
- version_conflict / duplicate_action
- timeout → 自動STAND
- **1対1のため必ず DEALER_TURN に遷移**

21.3 E2E

Phase1〜2

- **1 利用者接続でプレイ完了**
- 再接続復元

- 多重接続制御
- Dealer 伏せ札非公開
- rolling update 耐性
- deploy 後正常

Phase3

- Redis Pub/Sub 配信
- 複数インスタンス同期

21.4 負荷試験

- k6
- 5分負荷
- 100ユーザー
- reconnect混在

21.5 品質ゲート

- PR: unit + build + integration 必須
- main: smoke test 必須
- production: security NGなし
- 完成版 = `.github/workflows` が **18.3・8.12** を実コマンドで満たすこと。

21.6 受け入れ条件

- version は game_sessions.version のみ
- 再戦は READY を経由しない
- state schema 完全一致
- status が単一状態
- 三層防衛がPhase1で実装済
- multi-instance は Phase3

22. 非機能要件

項目	目標
WS遅延	p95 100ms以内
HTTP応答	p95 500ms以内
同時接続	100ユーザー
ルーム人数	**卓あたり人間 最大 1 名**
自動復元	即時
deploy	**GitHub Actions → Vercel 直接**
rollback	可能

22.1 計測前提

- staging、100 同接前提、HTTP/WS p95 の**サーバー側定義**、未達時は数値変更より先にチューニング。

23. 実装優先順位

Phase 1

- 認証
- ルーム
- WebSocket
- start
- hit / stand
- dealer
- result
- 三層防衛
- 自動 STAND
- 再接続

- **1 卓 1 人制約**
- **ディーラー AI + ライブラリ進行**

Phase 2

- rematch
- history
- 境界ケース

Phase 3

- Redis Pub/Sub
- multi-instance
- CI/CD 完全化
- 監視
- 負荷試験

---

24. 最終ルール

1. ブラックジャック計算は外部ライブラリ
2. **進行アルゴリズムはライブラリを UseCase 経由で適用する**
3. 正本は PostgreSQL
4. Redis は補助
5. WS は同期専用
6. 更新系は三層防衛必須
7. Dealer は 1 ドロー 1 Tx
8. 再接続は完全復元
9. CI 通過必須
10. **deploy は GitHub Actions から Vercel へ直接行う**
11. migration 失敗は中断

12. 未定義挙動は実装しない
13. **親（ディーラー）＝コンピューター、子（プレイヤー）＝利用者 1 対 1**
14. **ディーラーはヒューリスティック AI 基本戦略を用いる**
15. **AI とブラックジャックアルゴリズムの適用は UseCase に集約する**
16. ディーラーの Hit/Stand は §4.3・§6.1 を正とする
17. ライブラリは **提案であり最終決定ではない**